

TEKNOLOGI IOT II
Modul 5 (LATIHAN 5.6)



DISUSUN OLEH:

NAMA : DEA ANGGRAENI
NPM : 25782036
KELAS :3B
JURUSAN :TEKNOLOGI INFORMASI
PRODI :TEKNOLOGI REKAYASA
INTERNET

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, SPd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026/2027

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Sempul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

- Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
- Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
- Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawaban :



Analisis Log Pengiriman: Berdasarkan Serial Monitor, pengiriman paket ke berbagai perangkat berjalan sukses dengan status "Berhasil Diterima", namun langsung berubah menjadi "Gagal (Tidak Terjangkau)" ketika perangkat tujuan dimatikan.

Validasi Target MAC Address: Pengujian membuktikan bahwa kegagalan hanya terjadi secara spesifik pada baris MAC address perangkat yang kabel dayanya dicabut, sementara komunikasi ke perangkat lain yang masih aktif tetap berjalan normal.

Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4): Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawaban :

i. Waktu Deteksi Node Keluar (*Node Down*): Berdasarkan hasil observasi pada Serial Monitor, ketika salah satu board dimatikan atau tombol RST ditekan, fungsi *callback* `changedConnectionCallback()` membutuhkan waktu sekitar **3 hingga 7 detik** untuk mendeteksi hilangnya koneksi dan memperbarui topologi *mesh*.

ii. Waktu Penyambungan Kembali (*Self-Healing*): Saat daya board disambungkan kembali, node tersebut membutuhkan waktu sekitar **5 hingga 12 detik** untuk melakukan pemindaian jaringan (*scanning*) secara otonom dan kembali bergabung ke dalam rantai *mesh* tanpa perlu konfigurasi ulang.

```
setLogLevel: ERROR | STARTUP |
STARTUP: init(): 0
STARTUP: AP tcp server established on port 2222
Mesh Node [Node-1] Berjalan. Menunggu pembentukan topologi...
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.4, "kelembapan": 68.2}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.4, "kelembapan": 67.9}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.4, "kelembapan": 68.4}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.4, "kelembapan": 68.1}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.5, "kelembapan": 67.9}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.5, "kelembapan": 67.5}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.5, "kelembapan": 67.5}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.5, "kelembapan": 67.2}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.5, "kelembapan": 66.9}
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.5, "kelembapan": 66.6}
Mesh Node [Node-1] Berjalan. Menunggu pembentukan topologi...
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.9, "kelembapan": 65.6}
--> Koneksi Baru Terdeteksi! Node ID: 625338408
--> Topologi rantai mesh telah diperbarui
=====
[TERIMA DARI] Node-5 (Node ID: 625338408 | Chip ID: 625338408)
Suhu      : 32.30 °C
Kelembapan : 64.90 %
=====
[KIRIM MESH] {"node": "Node-1", "chipId": 625318081, "suhu": 32.8, "kelembapan": 65.6}
=====
[TERIMA DARI] Node-2 (Node ID: 3952257474 | Chip ID: 3952257474)
Suhu      : 31.80 °C
```

Deteksi Koneksi Node Baru: Terlihat baris log --> Koneksi Baru Terdeteksi! Node ID: 625338408 dan pembaruan topologi mesh yang menandakan bahwa Node-1 berhasil mendeteksi dan menjalin komunikasi nirkabel secara otonom dengan node lain di sekitarnya.

Penerimaan Data Telemetry Masuk: Terdapat blok log [TERIMA DARI] Node-5 (dengan *Chip ID* 625338408) dan Node-2 yang berhasil menerima data sensor dari node lain secara *real-time*, masing-masing menampilkan informasi parameter **Suhu** (contohnya 32.30 °C dan 31.80 °C) serta **Kelembapan** ruangan.

Komunikasi Multi-Arah Berjalan Normal: Selain menerima data dari node lain, Node-1 secara bersamaan tetap menjalankan tugas pengirimannya sendiri secara berkala melalui format [KIRIM MESH] yang berisi data suhu dan kelembapan lokalnya sendiri.

Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4): Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.

- i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetry dari Node <chipId> aman.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Jawaban :

sketch_oct5a.ino

```
1  #include <painlessMesh.h>
2  #include <DHT.h>
3  #include <ArduinoJson.h>
4
5  #define MESH_PREFIX    "iyaiyadeh"    // Nama jaringan Mesh bersama
6  #define MESH_PASSWORD "bismillah"    // Kata sandi jaringan Mesh
7  #define MESH_PORT      2222          // Port komunikasi TCP Mesh
8
9  // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2        // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22   // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25     // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
26     float suhu = dht.readTemperature();
27     float kelembapan = dht.readHumidity();
28
29     // Validasi pembacaan sensor
30     if (isnan(suhu) || isnan(kelembapan)) {
31         Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
32         return;
33     }
34
35     // Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
36     StaticJsonDocument<200> doc;
37     doc["node"] = nodeName;
38     doc["chipId"] = mesh.getNodeId();
39     doc["suhu"] = suhu;
40     doc["kelembapan"] = kelembapan;
41
42     String msg;
43     serializeJson(doc, msg);
44 }
```

sketch_oct5a.ino

```
45 // Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
46 mesh.sendBroadcast(msg);
47
48 Serial.print("[KIRIM MESH] ");
49 Serial.println(msg);
50 }
51
52 // Callback otomatis saat menerima pesan dari node manapun di jaringan mesh (Modifikasi Latihan 3)
53 void receivedCallback(uint32_t from, String &msg) {
54     // Parsing muatan teks JSON yang diterima
55     StaticJsonDocument<200> doc;
56     DeserializationError error = deserializeJson(doc, msg);
57
58     if (!error) {
59         const char* sender = doc["node"];
60         uint32_t chipId = doc["chipId"];
61         float suhu = doc["suhu"];
62         float kelembapan = doc["kelembapan"];
63
64         // Ambang batas suhu kritis (32.0 °C)
65         if (suhu > 32.0) {
66             // Jika suhu di atas 32.0 °C, cetak kartu telemetry penuh
67             Serial.println("=====");
68             Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
69             Serial.printf("Suhu : %.2f °C\n", suhu);
70             Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
71             Serial.println("=====");
72         } else {
73             // Jika suhu normal (<= 32.0 °C), cetak status singkat satu baris
74             Serial.printf("[NORMAL] Telemetry dari Node %u aman.\n", chipId);
75         }
76     } else {
77         Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
78     }
79 }
80
81
82 // Callback otomatis saat ada perangkat baru bergabung ke mesh
83 void newConnectionCallback(uint32_t nodeId) {
84     Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
85 }
86
87 // Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
88 void changedConnectionCallback() {
89     Serial.println("--> Topologi rantai mesh telah diperbarui");
90 }
```

```

89     Serial.println("Topologi jaringan mesh telah diperbarui.");
90 }
91
92 void setup() {
93     Serial.begin(115200);
94
95     // Inisialisasi sensor DHT
96     dht.begin();
97
98     // Atur level log debugging (hanya tampilkan error dan status startup)
99     mesh.setDebugMsgTypes(ERROR | STARTUP);
100
101     // Inisialisasi jaringan mesh
102     mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
103
104     // Daftarkan event callback
105     mesh.onReceive(&receivedCallback);
106     mesh.onNewConnection(&newConnectionCallback);
107     mesh.onChangedConnections(&changedConnectionCallback);
108
109     // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
110     userScheduler.addTask(taskSendMessage);
111     taskSendMessage.enable();
112
113     Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
114 }
115
116 void loop() {
117     // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
118     mesh.update();
119 }

```

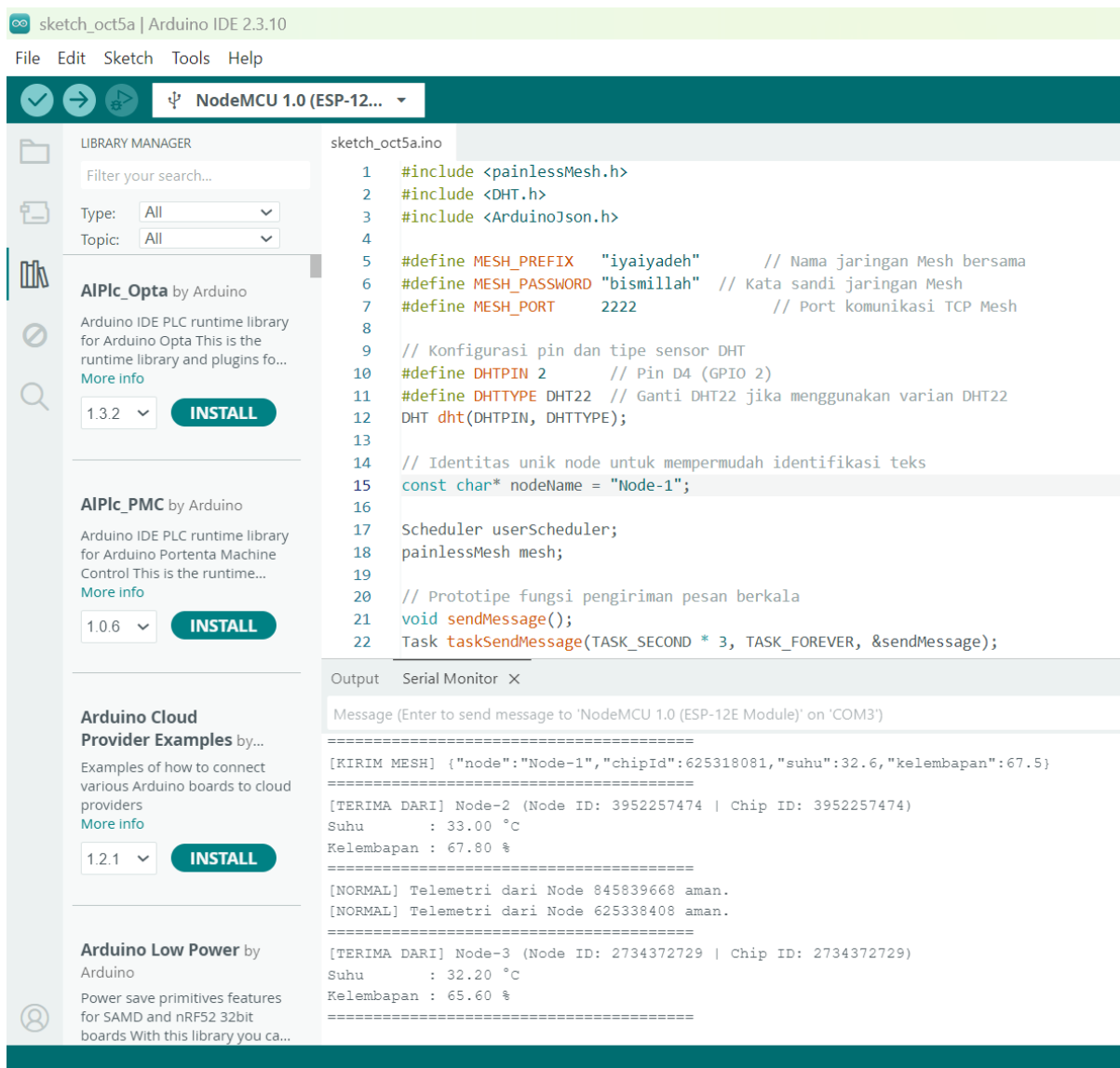
Konfigurasi Awal & Pustaka (#include & #define): Mengimpor pustaka utama seperti `pinlessMesh.h`, `DHT.h`, dan `ArduinoJson.h`, serta mendefinisikan parameter identitas jaringan (`MESH_PREFIX`, `MESH_PASSWORD`, `MESH_PORT`) dan pin sensor DHT22 pada GPIO 2.

Fungsi Pengiriman Berkala (sendMessage): Membaca data suhu dan kelembapan secara fisik dari sensor DHT, merangkumnya ke dalam format objek JSON (`StaticJsonDocument`), lalu menyiarkannya secara *broadcast* ke seluruh perangkat di dalam jaringan *mesh*.

Fungsi Pengawas Suhu Kritis (receivedCallback): Berfungsi menerima paket pesan masuk, melakukan *parsing* data JSON, dan memeriksa kondisi suhu menggunakan parameter `if (suhu > 32.0)`. Jika suhu melampaui batas kritis, program mencetak kartu telemetri penuh secara mencolok.

Format Satu Baris Ringkas (else): Jika suhu bernilai normal atau ≤ 32.0 °C, program menyaringnya dengan hanya mencetak status satu baris [NORMAL] Telemetri dari Node ... aman agar Serial Monitor tidak penuh oleh tumpukan data.

Manajemen Jaringan & Perulangan (setup & loop): Mengatur inisialisasi serial, pendaftaran fungsi *callback* event koneksi (*new* dan *changed*), penjadwalan tugas pengiriman telemetri, serta fungsi `mesh.update()` yang wajib dipanggil secara rutin untuk menjaga stabilitas topologi *mesh*.



Analisis Log Kondisi Kritis ($> 32.0\text{ }^{\circ}\text{C}$): Ketika node tetangga (seperti Node-2 dengan suhu $33.00\text{ }^{\circ}\text{C}$) melaporkan suhu di atas ambang batas kritis, program berhasil mengeksekusi blok kondisi if dan mencetak kartu telemetry penuh secara lengkap ke Serial Monitor.

Analisis Log Kondisi Normal ($\leq 32.0\text{ }^{\circ}\text{C}$): Sebaliknya, ketika node lain mengirimkan data telemetry dengan suhu yang aman atau normal, program secara otomatis melakukan penyaringan (*filtering*) dan hanya mencetak status singkat satu baris, seperti [NORMAL] Telemetry dari Node ... aman.

Kesimpulan Pengujian: Modifikasi fungsi received callback menggunakan *parser* JSON dan pengondisian ambang batas suhu terbukti berhasil mencegah penumpukan data (*spam*) di Serial Monitor, sehingga sistem pengawas kondisi lingkungan dapat berjalan lebih efektif dan efisien.